

Predicting a binary yes/no label on a 15-feature mixed-type table

G2604682C · Daniel Frutos Rodriguez

IN6227-Assignment-1 · Variant-2

Model: Claude Opus 5.5 (claude-opus-5-5) · Interface: Claude Code 2.1.292 · Skill: tabular-classification-report 1.0.0

Repository: <https://github.com/danielfrgz/in6227-assignment-1>

1. DATA EXPLORATION AND CLEANING

train.csv has 31,112 rows and 16 columns (comma-delimited, header row): 7 numeric, 7 categorical and 2 boolean columns, one of which is the target label (last column, two classes). After dropping the 3 rows with no label, training has 31,109 rows: no 23,645, yes 7,464 (minority share 0.240). The provided test.csv (same 16 columns) is the held-out test set, untouched until final evaluation (L4 p3): 13,333 labelled rows (no 10,165, yes 3,168); the largest class-share difference between the files is 0.002, so there is no distribution shift.

Quality issues. There are 0 exact duplicate rows and 0 test rows that duplicate a training row (L2 p29). Missing values are empty strings only, at most 0.022% of any column (brew_preference). Since this is under 5%, they are imputed with the median for numeric columns (L3 p10) and the mode for categorical ones (L3 p11), with no indicator columns. Imputers are fitted on training folds only (L4 p3). Possible disguised missing values were shown to the user, who chose to **keep them as real values**: an Unknown level in personal_interest (1,767 rows) and species (1,760); zeros shared by performance_score and stability_index (265 rows, Jaccard 1.0, all no) and by load_ratio and activity_duration (38 rows, no 32 / yes 6); and 35 rows at the maximum of aptitude_score (510), possibly a cap.

Outliers (Tukey fences): activity_duration has 5,350 rows outside the fences (0.172), load_ratio 2,527 (0.081), and the other numeric columns at most 0.036. They were kept: without task knowledge they cannot be shown to be noise (L2 p32, p60), and neither model is scale-sensitive. No value is negative, so none is impossible (L2 p8–10).

IDs and leakage. No column has a distinct-value ratio above 0.95, so none is ID-like (L2 p6). As a one-rule classifier (balanced accuracy, 3-fold CV), the best single feature is aptitude_score at 0.591, far below the 0.95 leak threshold. The column names are real words but form no coherent domain, so **the meaning of the columns is unknown, and the check of whether a value is known before the outcome could not be performed.**

2. FEATURE SELECTION AND ENGINEERING

All 15 features were kept, a deliberate finding. The highest single-value share is 0.897 (region), below the 0.99 near-constant cut (L2 p51); there are no duplicate columns and no numeric pair with $|r| > 0.95$. No column is skewed past $|2|$ (maximum 1.705, load_ratio) and there are no date or text columns, so nothing called for a new feature. PCA would cost interpretability (L2 p46–48), and building features from names of unknown meaning would be guesswork (L1 p54).

Encoding per model (L2 p59). Tree: categorical columns as ordinal codes (unseen levels $\rightarrow -1$), numeric as is. Naive Bayes: levels with $< 1\%$ of rows (17 levels of region, 4.8% of rows; 2 each in geological_era and mineral_type) and unseen levels go to a `__rare__` bucket; numeric columns become equal-frequency bins (L2 p56). No column has an evident order, so all are nominal (L2 p8).

As embedded selection (L2 p52), tree importances are led by weather_pattern (0.495) and aptitude_score (0.266), while region and geological_era (0.001 each) contribute almost nothing. weather_pattern scored only 0.5 alone in the one-rule screen, so its value appears only in combination with other features.

3. MODEL TRAINING

Baseline: always predict the majority class no (L5 p16).

Model 1: a CART tree with Gini (L4): scale-free, handles mixed types, insensitive to outliers, but with axis-parallel boundaries (L4 p66). **Model 2:** categorical Naive Bayes, chosen because 8 of the 15 features are categorical or boolean (share $0.533 \geq 0.50$). It is a high-bias model that assumes features are independent given the class (L5 p36), in contrast with the tree's low-bias, high-variance partitions (L5 p48). A random forest was not triggered: a depth-5 tree did not beat a full tree in CV (difference $-0.002 < 0.02$), and there are fewer than 50,000 rows.

Validation: 5-fold stratified CV, seed 42 (a setting, not a hyperparameter, L5 p10); 31,109 rows make 5 folds stable at modest cost (L4 p81). Minority share $0.240 \geq 0.20$, so no class weighting.

Search: an exhaustive grid scored by macro-F1. Tree: max_depth in {3, 5, 8, None} $\times$ min_samples_leaf in {1, 5, 20} $\times$ ccp_alpha in {0, 0.001}, 24 configurations; best max_depth 8, min_samples_leaf 20, ccp_alpha 0. Naive

Model	CV macro-F1	Test acc. [95% CI]	Test macro-F1	Rec./prec. yes	AUPRC	ROC-AUC
Baseline	0.432 $\pm$ 0.000	0.762 [0.755, 0.770]	0.433	0.000 / 0.000	0.238	0.500
Decision tree	0.747 $\pm$ 0.006	0.831 [0.824, 0.837]	0.745	0.529 / 0.687	0.659	0.872
Naive Bayes	0.755 $\pm$ 0.004	0.799 [0.793, 0.806]	0.756	0.796 / 0.554	0.686	0.880

Bayes: alpha in $\{0.1, 1\} \times$ bins in $\{5, 10\}$, 4 configurations; best alpha 0.1, 10 bins.

Stopping criteria. (1) Inside the model: tree pre-pruning (max_depth, min_samples_leaf, L4 p58) and post-pruning (ccp_alpha, L4 p59); the chosen min_samples_leaf = 20 is the grid's largest value. Naive Bayes is fitted by counting. (2) The search stops when the grid is exhausted. (3) Overfitting check: the training – CV macro-F1 gap is 0.014 (tree) and 0.000 (Naive Bayes), well under 0.10 (L4 p50–54).

The fitted tree has depth 8 and 173 leaves. Naive Bayes estimates 262 conditional probabilities (131 categories $\times$ 2 classes).

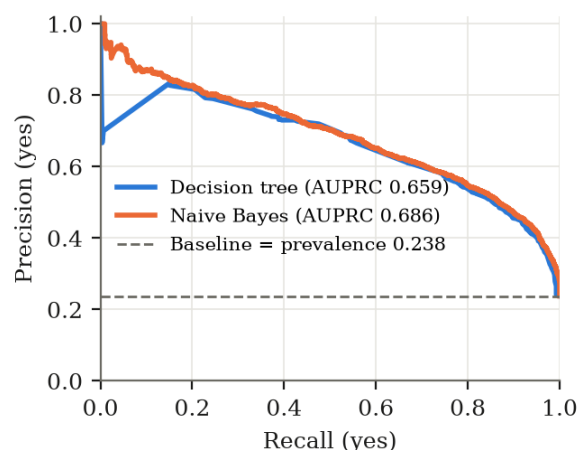
4. EVALUATION AND COMPARISON

Metric. Always predicting no scores accuracy 0.760 > random 0.500, so accuracy rewards exploiting the class distribution and is rejected (L4 p74). On macro-F1 that baseline scores 0.432 < random 0.464, so **macro-F1** (F-measure averaged over classes, L4 p77) is the main metric. The results table also gives accuracy with its Wilson 95% CI (L4 p87–88), AUPRC for yes (baseline = prevalence 0.238, L4 p84) and ROC-AUC (L4 p83); CV is mean $\pm$ sd.

Both models beat the baseline on every metric; on test macro-F1, tree – NB = -0.011 . A paired bootstrap (1,000 resamples of the 13,333 test rows, seed 42, both models scored on the same resample because they share the test rows, L4 p81, p91) gives a 95% CI for the difference of $[-0.020, -0.002]$. The interval excludes 0, so the difference is significant; the tree was better in only 0.008 of the resamples.

Decision tree		Naive Bayes	
True	Predicted	True	Predicted
no	9,403	8,136	2,029
yes	1,493	645	2,523

Test-set confusion matrices (rows: true class; columns: predicted).



Precision–recall curves for class yes on the test set.

5. FINDINGS AND DISCUSSION

Recommendation: Naive Bayes. The bootstrap CI excludes 0, so the main metric decides, and Occam's razor (L4 p56) is not invoked. The two models make different trade-offs. The tree has the higher accuracy (0.831 vs 0.799, with non-overlapping CIs) because it favours no: it finds 0.529 of the yes cases at precision 0.687. Naive Bayes finds 0.796 of them at precision 0.554 (2,523 vs 1,675 true positives, but 2,029 vs 762 false alarms). Its AUPRC (0.686 vs 0.659) and ROC-AUC (0.880 vs 0.872) are also higher, so its advantage does not depend only on the default 0.5 decision threshold. If false yes predictions cost much more than misses, a cost matrix (L4 p75) could favour the tree.

Neither model overfits much (gaps 0.014 and 0.000), and the CV scores (0.747 $\pm$ 0.006 vs 0.755 $\pm$ 0.004) agree with the test ranking. With depth 8 and 173 leaves, the tree is not "small" (L4 p45), so its interpretability is limited to its top splits.

Limitations. The meaning of the columns is unknown, so leakage was checked only statistically. Zeros and Unknown were kept as real values; the 265 training rows (122 test rows) with both performance_score and stability_index at 0 are all no, so if those zeros are "not recorded" codes linked to the outcome, both models profit in a way that may not carry over. The bootstrap captures test-sampling variation, not retraining variation (the CV sd gives that view). The tree's best min_samples_leaf is at the grid edge, and the significant difference is small (0.011), so stronger pruning or decision-threshold tuning, neither done here, could change the ranking.

1. HUMAN OVERSIGHT

I ran the skill three times. Two were development runs (classification-run-20261006-2339 on skill v0.1.0, and classification-run-20261007-0822 on v1.0.0). The third produced the submitted report (classification-run-20261007-2104, Claude Code 2.1.292, Opus 5.5, seed 42). All three ran in a folder outside my build workspace, so my own project notes could not load into the session and influence the output. I checked this at the start of each session.

I used manual mode in every session and approved each shell command myself. I declined the prompts to switch to automatic approval or to approve python commands in bulk. Claude Code can undo its own file edits but not the side effects of shell commands, so an unreviewed command cannot be rolled back. Approving each one also meant I read `train.py` before it ran. That is where I confirmed that imputation, rare-level grouping and binning are all fitted inside each cross-validation fold rather than on the full training set. If they were not, every number in the report would be optimistic and the evaluation would be invalid.

Most of my interventions happened in the development runs. At Checkpoint A of the first run, the skill proposed using `test.csv` as the held-out set based on its filename. I asked it to confirm the file actually contained a populated label column first, and to fall back to a stratified split if it did not. It did contain labels (13,334 rows; 10,165 no, 3,168 yes, 1 empty), so the decision stood, but it stood on evidence rather than on a filename.

At Checkpoint A of the second run I corrected two things. The skill described the co-occurring zeros as looking like a not-recorded code. That is an inference rather than a measurement, so I asked it to report only what it had measured: perfect co-occurrence in both column pairs (Jaccard 1.0), and zero falling outside the Tukey fences in three of the four columns. It also proposed a Claude Code version read from `claude --version`, which reports the installed binary rather than the running session. I corrected it to the running version.

In the submitted run I approved the plan at Checkpoint B and the draft at Checkpoint C without changes, and the run log records no overrides of skill defaults. The rules I would have overridden had already been corrected in the skill after the earlier runs. I still answered the Checkpoint A questions myself: the target column, the held-out file, the disguised-missing candidates and the interface version.

What I was checking for at each checkpoint was whether the approach followed the course material, and whether each choice was justified by something the skill had actually measured. I am not sure what I would do differently next time.

2. CRITICAL EVALUATION

The decision I most want to challenge is one of my own rules.

Rule G10 chose the primary metric from the minority class share. At 0.20 or above it selected accuracy. The measured share was 0.240, so the rule fired as written. It was still the wrong metric. The majority-class baseline scores 0.760 accuracy without identifying a single yes, so tuning on accuracy rewards configurations that predict the majority class. Class share is a proxy for how much room a metric has above a trivial model, and on this data the proxy and the thing it stands for came apart.

I overrode the rule at Checkpoint B of the first development run. I switched tuning and the headline metric to macro-F1, added AUPRC, and replaced the ROC curves with precision-recall curves (L4 p84). The override changed the result. The decision tree wins on accuracy (0.831 against 0.799, with non-overlapping Wilson intervals). Naive Bayes wins on macro-F1 (0.756 against 0.745) and on AUPRC (0.686 against 0.659), and the paired bootstrap of the macro-F1 difference is $[-0.020, -0.002]$, which excludes zero. Under the original rule I would have recommended the tree. The per-class recall shows the mechanism: the tree recovers 0.529 of the yes class, Naive Bayes 0.796.

I agree with the override. L4 p74 shows that accuracy is misleading when one class dominates, which is the situation here, so macro-F1 was the choice the course material supports.

I replaced the rule rather than leaving it in place and overriding it again. The new rule rejects a primary metric that the majority-class baseline already scores well on. The skill then corrected my phrasing. Read literally, my version always selects macro-F1, because a majority baseline's macro-F1 is always lower than its accuracy. It changed the test to a comparison against a random guesser. That gives the same answer here (accuracy 0.760 against 0.500, rejected; macro-F1 0.432 against 0.464, accepted) and still allows accuracy to be chosen on balanced data. In the submitted run the rule selected macro-F1 on its own, with no override anywhere in the plan.

This was the second time the skill corrected me instead of complying. The first was at Checkpoint C of the first development run, where I asked for `ccp_alpha` to be listed as a pre-pruning stopping criterion and it pointed out that cost-complexity pruning is post-pruning (L4 p59).

The report still contains an inference of the kind I corrected at Checkpoint A. Section 3 says the fully grown tree memorises noise, when what was measured is that it fits the training data perfectly and gains nothing in cross-validation. It means there are still improvements to be made.

One inconsistency I chose not to resolve. Rule G5 still decides class weighting from a bare 0.20 share threshold, which is the same form of rule I replaced in G10. At a share of 0.240 the output is defensible either way, and the argument for class weighting is much weaker than the argument against accuracy as a tuning metric. What is inconsistent is the form of the rule, not its result on this data. I recorded it rather than changing an untested rule hours before the deadline.

A second limitation I recorded rather than fixed. The tuned tree's best `min_samples_leaf` (20) sits at the edge of the search grid, so the optimum may lie outside it. Extending the grid after seeing that result would have been a change driven by the outcome, so the grid stands as specified and the report states the limitation.

3. TRUSTWORTHINESS

I verified the report by recomputing it rather than by rereading it.

I worked in a separate Python session, outside the one that produced the results. I recomputed accuracy for all three models from the confusion matrices printed in the report: 0.762, 0.831 and 0.799, all matching. Two further checks come from first principles rather than from the same library, so they test the numbers instead of repeating them. A majority-class predictor's macro-F1 must be $2 \times 0.762 / 1.762$, halved, which is 0.433; the report states 0.433. A constant classifier's AUPRC must equal the positive prevalence, $3,168 / 13,333 = 0.238$; the report states 0.238.

I also recomputed macro-F1 from the raw confusion-matrix cell counts, and got it wrong. I recorded 0.7454 and 0.7563 to four decimal places after feeding three-decimal precision and recall into the F1 formula. At full precision the values are 0.74531 and 0.75624, which match `metrics.json`. The three-decimal figures in the report were correct throughout. The error was only caught because the check was repeated independently afterwards.

The skill checks one of its own implementations instead of trusting it. The Wilson confidence interval is asserted against the worked example in the slides ($N = 100$, accuracy = 0.8, giving [0.711, 0.867], L4 p88). Determinism was confirmed three times in the submitted run: `train.py` was re-run twice after edits that only moved a figure legend, and `metrics.json` was identical each time apart from timing fields. The test set was therefore scored three times with identical code and identical results, and no decision was changed because of a test result. In an earlier build, all 47 numerical claims in the report were traced back to the specific key in `metrics.json` or `profile.json` they come from, with no mismatches.

I rate the confidence intervals as the strongest evidence here. I rate my own manual checks as weaker, because one of them contained an error caused by my inexperience with the precision involved.

What I could not verify is the meaning of the columns. The names are real words but do not describe a coherent domain, so no timing-based leakage check was possible, and the report says so. This is also why the zeros question cannot be settled. The pattern is consistent with a not-recorded code, with 265 training rows and 122 test rows where `performance_score` and `stability_index` are both zero and all labelled no, but the data cannot distinguish that from genuine zeros. Imputing them would replace recorded values with estimates on an assumption I cannot check.